TABLE OF CONTENTS

	Content	Page No.
1.	**Concept of the project**	**4**
1.1	Introduction to project	4
1.2	Motivation	5
1.3	Problem Statement	5
1.4	Objectives	5
1.5	Literature survey	6
1.6	Areas of Application	7
1.7	Benefits	7
2.	**Design of the project**	**8**
2.1	Methodology	8
2.2	Block Diagram	9
2.3	MATLAB Flowchart	9
2.4	MATLAB Algorithm	10
2.5	Verilog State Diagram	10
2.6	Verilog Algorithm	11
2.6.1	Verilog (parameter.v) Algorithm	11
2.6.2	Verilog (read.v) Algorithm	11
2.6.3	Verilog (write.v) Algorithm	12
2.6.4	Verilog (tb_simulation) Algorithm	12
3.	**Implementation of the project**	**13**
3.1	Components/tools used	13
3.2	MATLAB Code	14
3.3	Verilog Code	15
4.	**Results of the project**	**27**
4.1	Brightness Decrease Operation	27
4.2	Brightness Increase Operation	28
4.3	Invert Operation	29
4.4	Threshold Operation	30
5.	**Conclusion**	**31**

CHAPTER - 1
1. CONCEPT OF THE PROJECT

1.1 Introduction to project

Digital Image Processing involves the manipulation of images using algorithms and mathematical operations to extract information or enhance visual quality. Combining MATLAB and Verilog for image processing provides a comprehensive approach, leveraging MATLAB's powerful image processing toolbox for algorithm development and Verilog for hardware acceleration on FPGAs.

Our project centers around the utilization of Field-Programmable Gate Arrays (FPGAs) and Verilog programming to implement real-time image processing operations. FPGAs offer a versatile and efficient hardware platform that allows for the acceleration of image processing tasks, mimicking the functionality of an image sensor or camera.

To read the .bmp image on in Verilog, the image is required to be converted from the bitmap format to the hexadecimal format. We have written a code in MATLAB to convert a bitmap image to a .hex file. The input image size is 768x512 and the image .hex file includes R, G, B data of the bitmap image.

In this FPGA Verilog project, some simple processing operations are implemented in Verilog such as inversion, brightness control, threshold and black and white operations. The image processing operation is selected by a "parameter.v" file and then, the processed image data are written to a bitmap image output.bmp for verification purposes. The image reading Verilog code operates as a Verilog model of an image sensor/ camera, which can be helpful for functional verifications in real-time FPGA image processing projects. The image writing part is also extremely useful for testing as well when you want to see the output image in BMP format.

The core concept of our project lies in the integration of fundamental image processing operations directly into the FPGA Verilog model. These operations are carefully selected to encompass essential techniques in image manipulation. The primary image processing operations implemented are:

- **Inversion:** In image processing, inversion involves transforming pixel intensities to their complementary values, resulting in a reversal of the brightness levels in the image.

- **Brightness Control:** Adjusting the overall brightness of an image by uniformly scaling pixel intensities, enhancing or dimming the visual appearance.

- **Threshold Operations:** Segmenting an image by converting pixel intensities to binary values based on a specified threshold, separating regions with intensities above or below the threshold.

- **Black and White Operation:** Transforming a color image into a grayscale format, where each pixel is represented solely by its luminance value, removing color information.

1.2 Motivation

The surging demand for real-time image processing is propelled by the critical needs of domains like medical diagnostics, surveillance, and multimedia applications. Conventional methods grapple with the dual challenge of attaining high-speed processing without compromising flexibility, creating a bottleneck for dynamic applications. Recognizing this, our project takes a pioneering approach by seamlessly amalgamating Verilog for robust hardware acceleration and MATLAB for nuanced algorithm development. This synthesis not only promises to overcome the limitations of traditional methods but also seeks to redefine the landscape of real-time image processing. By uniting the precision of Verilog with the versatility of MATLAB, our endeavor aspires to not merely meet demands but to usher in a new era of innovation and efficiency in the realm of image processing.

1.3 Problem Statement

In the realm of image processing, there is an increasing demand for systems that can efficiently handle large volumes of image data in real-time while providing flexibility for algorithm development. Traditional software-based approaches often face limitations in terms of processing speed, particularly when dealing with high-resolution images. This project seeks to bridge the gap by combining Verilog, a hardware description language, and MATLAB, a high-level scripting language, to create a versatile and high-performance image processing system.

The successful implementation of this project is expected to contribute to the advancement of image processing techniques, offering a versatile solution that can be applied across various domains, including medical imaging, surveillance, and multimedia processing.

1.4 Objectives

The main objective of this project is to develop a system in which we can process the images and convert it as per our needs. The primary goals are as follows:

- Real-Time Image Enhancement: Develop FPGA-based algorithms to enhance the quality of images in real-time, including tasks like noise reduction, colour correction, and contrast enhancement.
- Real-Time Object Detection: Implement real-time object detection algorithms on FPGA to identify specific objects or features within images, such as faces, objects, or patterns.
- Video Compression: Create FPGA-based video compression algorithms to reduce the data size of video streams, suitable for video transmission and storage applications.
- Low-Power and Cost-Efficient Solutions: Optimize FPGA designs for low power consumption and efficient resource utilization, allowing for cost-effective and energy-efficient image processing solutions

1.5 Literature Survey

Title	Authors	Publication year	Source	Summary
Image Enhancement Methods Approach Using Verilog Hardware Description Language	Chiuchisan, M. Cerlinca	2012	International Conference on DEVELOPMENT AND APPLICATION SYSTEMS	The introduction highlights the crucial role of digital image processing in performance improvement, advocating for an innovative, Verilog based hardware implementation approach.
A Review on Image Enhancement using Hardware Co-simulation for Biomedical Application	Kalyani A. Dakre,	2014	International Journal of Advanced Research in Computer Engineering & Technology(IJARCET)	This section emphasizes FPGA superiority over DSPs for real-time image enhancement using Verilog, MATLAB, and System Generator.
An Implementation of Image Enhancement on Real Time Configurable system using HDL	Mahavir Singh, Gitanjali Pandove	2018	International Journal of Advanced Research in Electronics and Communication engineering	This part highlights image quality issues during transformations, advocating hardware for superior performance with VLSI and HDL solutions.
Image Enhancement on FPGA using Verilog	Dr. Sagar Patel, Krinesh Patel, Keval Patel and Chaitanya Patel	2019	International Journal of Technical Innovation in Modern Engineering &Science (IJTIMES)	Explores image magnification's role, emphasizing FPGA based enhancement with Verilog and MATLAB for diverse applications.

1.6 Areas of Applications

- Medical Imaging: Critical for diagnostics and treatment planning, including techniques like X-ray, MRI, and CT image analysis.
- Remote Sensing: Crucial for environmental monitoring, agriculture, and urban planning through the analysis of satellite and aerial imagery.
- Security and Surveillance: Utilized for object recognition, tracking, and anomaly detection in video surveillance systems.
- Industrial Inspection: Essential for quality control, defect detection, and process monitoring in manufacturing and production environments.
- Automotive: Image processing is integral for driver assistance systems, autonomous vehicles, and traffic monitoring, contributing to enhanced safety and efficiency.

1.7 Benefits

- Improved Image Quality: Enhances clarity and accuracy by reducing noise, correcting distortions, and improving contrast in visual data.
- Automated Analysis: Enables efficient and accurate automated analysis of large datasets, aiding tasks such as object recognition, pattern detection, and classification.
- Medical Diagnosis: Facilitates precise medical imaging for accurate diagnosis, treatment planning, and non-invasive visualization, leading to improved healthcare outcomes.
- Enhanced Security and Surveillance: Contributes to advanced security systems with features like facial recognition, object tracking, and real-time anomaly detection.
- Efficient Data Handling: Supports efficient storage and transmission through compression techniques, minimizing storage space and bandwidth requirements, crucial for various applications.

CHAPTER – 2

DESIGN OF THE PROJECT

2.1 Methodology

MATLAB-Based Hexadecimal Conversion:

• Hexadecimal Conversion:

- Use MATLAB to convert the image data into hexadecimal format, ensuring proper representation for FPGA processing.
- Implement MATLAB scripts to convert pixel values to hexadecimal.

Verilog-Based Image Processing Operations:

- Image Data Integration in Verilog:
  - Receive the hexadecimal image data in Verilog modules.
  - Implement a Verilog interface to read and process the hexadecimal values.
- 3.Inversion Operation in Verilog:
  - Develop a Verilog module for inverting pixel values.
  - Utilize Verilog conditional statements to perform inversion.
- Brightness Control in Verilog:
  - Implement Verilog modules for adjusting brightness based on a specified algorithm or input parameters.
  - Ensure that the Verilog code preserves image quality during brightness adjustments.
- Threshold Operations in Verilog:
  - Create Verilog modules for applying threshold operations. o Utilize conditional statements in Verilog to determine pixel values based on a specified threshold.
- Black and White Conversion in Verilog:
  - Develop Verilog modules for converting the image to black and white.
  - Use appropriate Verilog logic to determine the black and white representation of pixels.

This adapted methodology focuses on the specific image processing operations you perform, integrating MATLAB for hexadecimal conversion and Verilog for FPGA-based implementation of inversion, brightness control, threshold operations, and black and white conversion.

2.2 Block Diagram

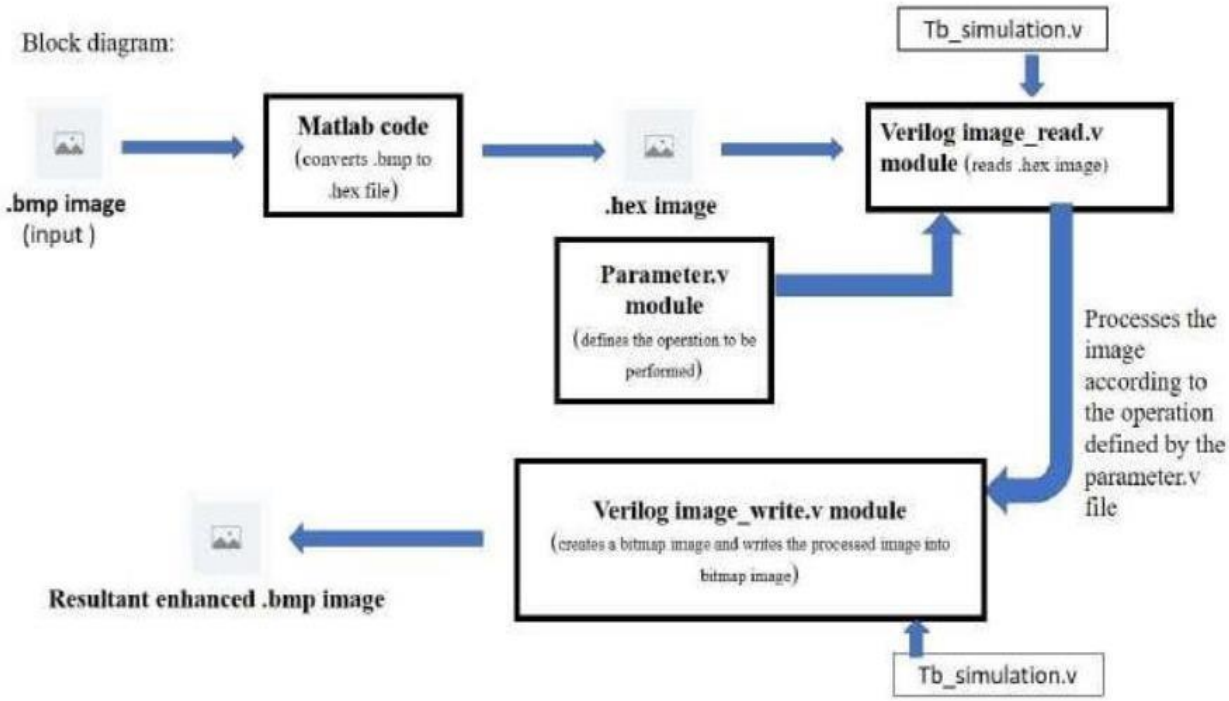


Fig 1. Block Diagram

2.3 MATLAB flowchart

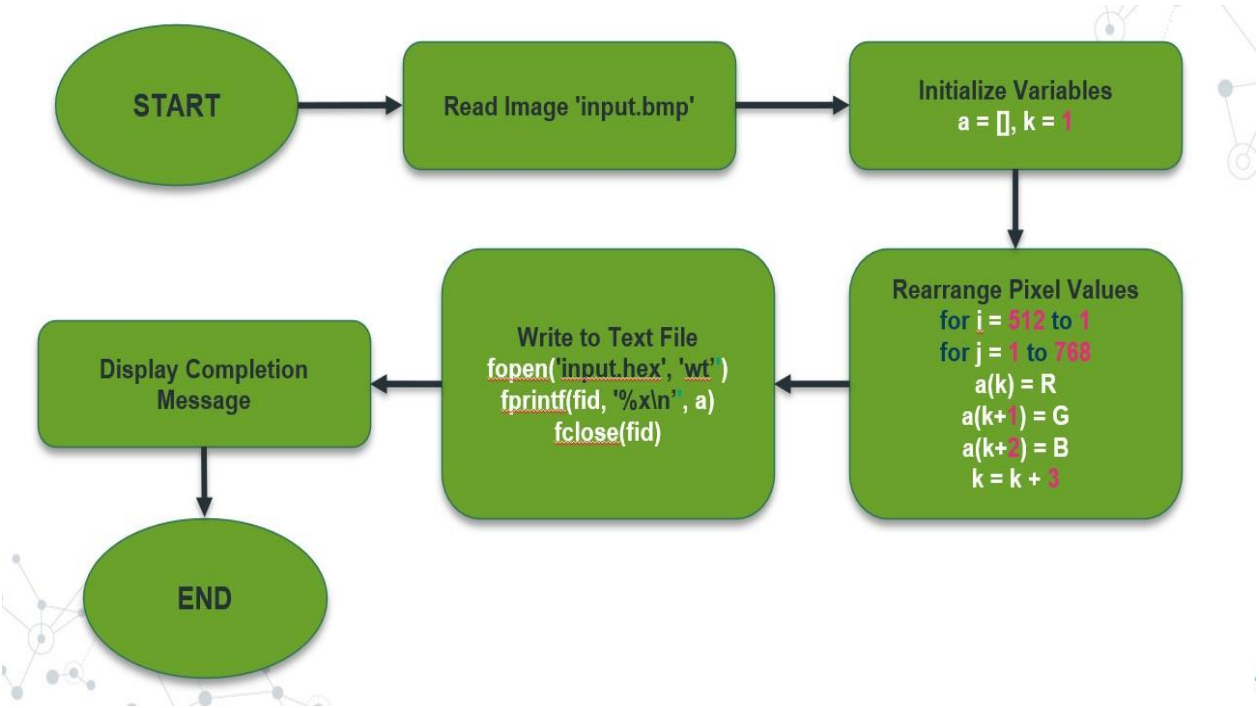


Fig 2 . MATLAB Flowchart

2.4 MATLAB Algorithm

1. Read Image:

- Read a 24-bit BMP image named 'input.bmp' using the **imread** function.

2. Initialize Variables:

- Initialize an empty array a to store rearranged pixel values.
- Initialize a counter k to keep track of the current index in the array.

3. Rearrange Pixel Values:

- Loop over the rows of the image from the last row to the first row.
- For each pixel in the row, extract the red, green, and blue values (RGB).
- Store the RGB values in the array a in the order: Red, Green, Blue.
- Increment the counter k by 3 for each pixel (since there are three colour channels).

4. Write to Text File:

- Open a text file named **'input.hex'** for writing using **fopen.**
- Write the values in the array a to the text file in hexadecimal format using **fprintf.**
- Close the text file using **fclose**

5. Display Completion Message:

- Display a message indicating that the text file write is complete.

The resulting **'input.hex'** file contains the rearranged pixel values of the input image in hexadecimal format. The code assumes the input image has dimensions 512x768 pixels, and it writes the values in reverse order from the last row to the first row.

2.5 Verilog State Diagram

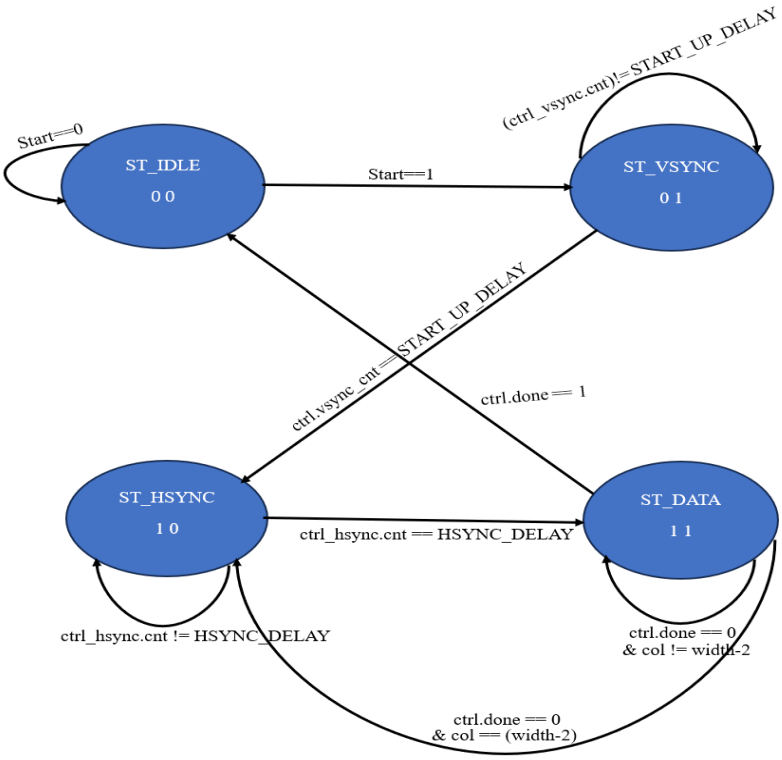


Fig 3 . Verilog State Diagram

2.6 Verilog Algorithm

2.6.1 Verilog (parameter.v) Algorithm

1. **File Names Definition:**
   - Define the input and output file names using preprocessor directives.
2. **Operation Selection:**
   - Choose the image processing operation by commenting or uncommenting the corresponding preprocessor directives.
   - Options include: Brightness operation , Invert operation , Black and white operation and Threshold operations.

2.6.2 Verilog (image_read.v) Algorithm

1. **Initialization and Parameters:**
   a. Initialize parameters for image dimensions, input file, delays, and image processing values.
   b. Read the image data from a hexadecimal file (**input.hex**) into a local memory array.
2. **Finite State Machine (FSM):**
   a. Implement an FSM with states for IDLE, VSYNC, HSYNC, and DATA processing.
   b. Create control signals for VSYNC, HSYNC, and data processing based on the FSM.
3. **Image Data Processing:**
   a. Process image data based on the specified operations: brightness addition/subtraction, inversion, black and white conversion, and thresholding.

   b. Implement pixel-wise operations for Red, Green and Blue components.
4. **Counters and Indexing:**
   a. Use counters for VSYNC, HSYNC, and data processing time periods.
   b. Index rows and columns for reading memory based on the current state.
5. **Image File Reading and Storage:**
   a. Read image data from the file during the start signal, converting it into temporary storage arrays for R, G, and B components.
6. **Output Generation:**
   a. Output processed pixel values for two pixels in parallel (DATA_R0, DATA_G0, DATA_B0, DATA_R1, DATA_G1, DATA_B1).
7. **Image Enhancement Operations (Conditional):**
   a. Conditionally execute brightness addition/subtraction, inversion, black and white conversion, or thresholding based on preprocessor directives.
8. **Done Signal:**
   a. Set a **CTRL done** flag to indicate completion of data processing for the entire image.

This Verilog module integrates image file reading, real-time processing, and various image enhancement operations, providing flexibility for different FPGA-based applications.

2.6.3 Verilog (image_write.v) Algorithm

1. **Header Initialization:**
   - Initialize a 54-byte BMP header with appropriate values, such as file type, image dimensions, and colour depth.
2. **Temporary Memory and Counters:**
   - Create a temporary memory array (**out_BMP**) to store the processed image data in RGB888 format.
   - Implement counters (**l** and **m**) to index rows and columns of the temporary memory.
3. **Image Data Writing:**
   - On each rising edge of the clock, write RGB888 data for both odd and even pixels to the temporary memory during HSYNC pulses.
4. **Data Counting:**
   - Count the number of pixels processed using the **data_count** variable during each HSYNC pulse.
5. **Done Signal:**
   - Set a **done** flag to indicate completion once all pixels are processed.
6. **File Writing:**
   - Open a BMP file in write-binary mode.
   - Once processing is complete (**Write_Done** is asserted), write the BMP header to the file.
   - Write the processed image data to the file in chunks of 6 bytes (R0, G0, B0, R1, G1, B1) in a loop.
7. **Write Completion:**
   - Set the **Write Done** signal to 1 when the entire image data is written to the file.
8. **File Closure:**
   - Close the BMP file after writing is complete

This Verilog module provides the functionality to write processed image data to a BMP file and ensures proper formatting with the BMP header.

2.6.4 Verilog (tb_simulation.) Algorithm

1. Timescale and Include Statement:

- Set the timescale to 1ns/1ps.
- Include the definition file (`parameter.v`).

2. Testbench Module (`tb_simulation`):

- Declare testbench module (`tb_simulation`).
- Declare signals: `HCLK`, `HRESETn`, `vsync`, `hsync`, and pixel data signals.
- Instantiate `image_read` and `image_write` modules with appropriate configurations.
- Use a clock generator for `HCLK`.
- Initialize `HRESETn` and assert it after a delay.

CHAPTER – 3 IMPLEMENTATION OF THE PROJECT

3.1 Components/tools used

HARDWARE TOOLS USED :

- FPGA Board(Spartan 6/7)

SOFTWARE TOOLS USED :

- **MATLAB :** It is a high-level programming language and interactive environment widely used for numerical computing, data analysis, and visualization. It provides a comprehensive set of tools and functions that make it particularly well-suited for tasks involving matrix operations, linear algebra, signal processing, and image analysis.

- **Xilinx Integrated Software Environment :** It is a comprehensive software suite designed for the development, testing, and implementation of digital systems on Xilinx. It is a comprehensive software suite tailored for Verilog-based FPGA development. It facilitates design entry, simulation, synthesis, and implementation processes, ensuring seamless integration of Verilog code into Xilinx FPGAs.

Fig4. FPGA Board

Fig 5. MATLAB Software

Fig 6. Xilinx Software

3.2 MATLAB Code

```
close all;
clear all;
clc;

b=imread('input.bmp'); % 24-bit BMP image RGB888

k=1;
for i=512:-1:1 % image is written from the last row to the first row
for j=1:768
a(k)=b(i,j,1);
a(k+1)=b(i,j,2);
a(k+2)=b(i,j,3);
k=k+3;
end
end
fid = fopen('input.hex', 'wt');
fprintf(fid, '%x\n', a);
disp('Text file write done');disp(' ');
fclose(fid);
```

3.3 Verilog Code

To read the image hexadecimal data file, Verilog uses this command: $readmemh or $readmemb if the image data is in a binary text file. After reading the image .hex file, the RGB image data are saved into memory and read out for processing.

```
/*************   Module for reading and processing image   ***********/
/*****************************************************************/
`include "parameter.v"       // Include definition file
// FPGA project: Image processing in Verilog
module image_read
#(
  parameter     WIDTH  = 768,   // Image width
                HEIGHT  = 512,  // Image height
  INFILE  = "input.hex",        // image file
  START_UP_DELAY = 100, // Delay during start up time
  HSYNC_DELAY = 160,// Delay between HSYNC pulses
  VALUE= 100, // value for Brightness operation
  THRESHOLD= 90, // Threshold value for Threshold operation
  SIGN=1         // Sign value using for brightness operation
  // SIGN = 0: Brightness subtraction
  // SIGN = 1: Brightness addition
)
(
  input HCLK,          // clock
  input HRESETn,         // Reset (active low)
  output VSYNC,        // Vertical synchronous pulse
 // This signal is often a way to indicate that one entire image is
transmitted.
 // Just create and is not used, will be used once a video or many images
are transmitted.
  output reg HSYNC,        // Horizontal synchronous pulse
 // An HSYNC indicates that one line of the image is transmitted.
 // Used to be a horizontal synchronous signal for writing bmp file.
  output reg [7:0]  DATA_R0,  // 8 bit Red data (even)
  output reg [7:0]  DATA_G0,  // 8 bit Green data (even)
  output reg [7:0]  DATA_B0,  // 8 bit Blue data (even)
  output reg [7:0]  DATA_R1,  // 8 bit Red  data (odd)
  output reg [7:0]  DATA_G1,  // 8 bit Green data (odd)
  output reg [7:0]  DATA_B1,  // 8 bit Blue data (odd)
  // Process and transmit 2 pixels in parallel to make the process faster,
you can modify to transmit 1 pixels or more if needed
  output     ctrl_done     // Done flag
);
//-------------------------------------------------
// Internal Signals
//-------------------------------------------------
parameter sizeOfWidth = 8;   // data width
parameter sizeOfLengthReal = 1179648;// image data : 1179648 bytes: 512
* 768 *3
// local parameters for FSM
```

```
localparam  ST_IDLE  = 2'b00,// idle state
   ST_VSYNC = 2'b01,// state for creating vsync
   ST_HSYNC = 2'b10,// state for creating hsync
   ST_DATA  = 2'b11;// state for data processing
reg [1:0] cstate,   // current state
nstate;    // next state
reg start;        // start signal: trigger Finite state machine beginning
to operate
reg HRESETn_d;   // delayed reset signal: use to create start signal
reg   ctrl_vsync_run; // control signal for vsync counter
reg [8:0] ctrl_vsync_cnt; // counter for vsync
reg   ctrl_hsync_run; // control signal for hsync counter
reg [8:0] ctrl_hsync_cnt; // counter  for hsync
reg   ctrl_data_run; // control signal for data processing
reg [7:0] total_memory [0 : sizeOfLengthReal-1];// memory to store  8-
bit data image
// temporary memory to save image data : size will be WIDTH*HEIGHT*3
integer temp_BMP   [0 : WIDTH*HEIGHT*3 - 1];
integer org_R   [0 : WIDTH*HEIGHT - 1];   // temporary storage for R
component
integer org_G   [0 : WIDTH*HEIGHT - 1]; // temporary storage for G
component
integer org_B   [0 : WIDTH*HEIGHT - 1]; // temporary storage for B
component
// counting variables
integer i, j;
// temporary signals for calculation: details in the paper.
integer    tempR0,tempR1,tempG0,tempG1,tempB0,tempB1;    //    temporary
variables in contrast and brightness operation

integer value,value1,value2,value4;// temporary variables in invert and
threshold operation
reg [ 9:0] row; // row index of the image
reg [10:0] col; // column index of the image
reg [18:0] data_count; // data counting for entire pixels of the image
//-----------------------------------------------//
// -------- Reading data from input file ----------//
//-----------------------------------------------//
initial begin
    $readmemh(INFILE,total_memory,0,sizeOfLengthReal-1);  //  read  file
from INFILE
end
// use 3 intermediate signals RGB to save image data
always@(start) begin
    if(start == 1'b1) begin
        for(i=0; i<WIDTH*HEIGHT*3 ; i=i+1) begin
            temp_BMP[i] = total_memory[i+0][7:0];
        end

        for(i=0; i<HEIGHT; i=i+1) begin
```

```
            for(j=0; j<WIDTH; j=j+1) begin
     // Matlab code writes image from the last row to the first row
     // Verilog code does the same in reading to correctly save image
pixels into 3 separate RGB mem
                org_R[WIDTH*i+j]      =      temp_BMP[WIDTH*3*(HEIGHT-i-
1)+3*j+0]; // save Red component
                org_G[WIDTH*i+j]      =      temp_BMP[WIDTH*3*(HEIGHT-i-
1)+3*j+1];// save Green component
                org_B[WIDTH*i+j]      =      temp_BMP[WIDTH*3*(HEIGHT-i-
1)+3*j+2];// save Blue component
            end
        end
    end
end
//-----------------------------------------------------//
// ---Begin to read image file once reset was high ---//
// ---by creating a starting pulse (start)------------//
//-----------------------------------------------------//
always@(posedge HCLK, negedge HRESETn)
begin
    if(!HRESETn) begin
        start <= 0;
  HRESETn_d <= 0;
    end
    else begin           //
        HRESETn_d <= HRESETn;       //        |  |
  if(HRESETn == 1'b1 && HRESETn_d == 1'b0)   //___0___| 1 |___0___ :
starting pulse
   start <= 1'b1;
  else
   start <= 1'b0;
    end
end
//-------------------------------------------------------------------//
// Finite state machine for reading RGB888 data from memory and creating
hsync and vsync pulses --//
//-------------------------------------------------------------------//
always@(posedge HCLK, negedge HRESETn)
begin
    if(~HRESETn) begin
        cstate <= ST_IDLE;
    end
    else begin
        cstate <= nstate; // update next state
    end
end
//-----------------------------------------//
//--------- State Transition --------------//
//-----------------------------------------//
// IDLE . VSYNC . HSYNC . DATA
```

```
always @(*) begin
 case(cstate)
  ST_IDLE: begin
   if(start)
    nstate = ST_VSYNC;
   else
    nstate = ST_IDLE;
  end
  ST_VSYNC: begin
   if(ctrl_vsync_cnt == START_UP_DELAY)
    nstate = ST_HSYNC;
   else
    nstate = ST_VSYNC;
  end
  ST_HSYNC: begin
   if(ctrl_hsync_cnt == HSYNC_DELAY)
    nstate = ST_DATA;
   else
    nstate = ST_HSYNC;
  end
  ST_DATA: begin
   if(ctrl_done)
    nstate = ST_IDLE;
   else begin
    if(col == WIDTH - 2)
     nstate = ST_HSYNC;
    else
     nstate = ST_DATA;
   end
  end
 endcase
end
// ------------------------------------------------------------------ //
// --- counting for time period of vsync, hsync, data processing ---  //
// ------------------------------------------------------------------ //
always @(*) begin
 ctrl_vsync_run = 0;
 ctrl_hsync_run = 0;
 ctrl_data_run  = 0;
 case(cstate)
  ST_VSYNC:  begin ctrl_vsync_run = 1; end  // trigger counting for vsync
  ST_HSYNC:  begin ctrl_hsync_run = 1; end // trigger counting for hsync
  ST_DATA:  begin ctrl_data_run  = 1; end // trigger counting for data
processing
 endcase
end
// counters for vsync, hsync
always@(posedge HCLK, negedge HRESETn)
begin
    if(~HRESETn) begin
```

```
        ctrl_vsync_cnt <= 0;
  ctrl_hsync_cnt <= 0;
    end
    else begin
        if(ctrl_vsync_run)
   ctrl_vsync_cnt <= ctrl_vsync_cnt + 1; // counting for vsync
  else
   ctrl_vsync_cnt <= 0;

        if(ctrl_hsync_run)
   ctrl_hsync_cnt <= ctrl_hsync_cnt + 1; // counting for hsync
  else
   ctrl_hsync_cnt <= 0;
    end
end
// counting column and row index  for reading memory
always@(posedge HCLK, negedge HRESETn)
begin
    if(~HRESETn) begin
        row <= 0;
  col <= 0;
    end
 else begin
  if(ctrl_data_run) begin
   if(col == WIDTH - 2) begin
    row <= row + 1;
   end
   if(col == WIDTH - 2)
    col <= 0;
   else
    col <= col + 2; // reading 2 pixels in parallel
  end
 end
end
//-------------------------------------------------//
//----------------Data counting--------------------//
//-------------------------------------------------//
always@(posedge HCLK, negedge HRESETn)
begin
    if(~HRESETn) begin
        data_count <= 0;
    end
    else begin
        if(ctrl_data_run)
   data_count <= data_count + 1;
    end
end
assign VSYNC = ctrl_vsync_run;
assign ctrl_done = (data_count == 196607)? 1'b1: 1'b0; // done flag
//-------------------------------------------------//
```

```
//-------------  Image processing   ---------------//
//-----------------------------------------------//
always @(*) begin

 HSYNC   = 1'b0;
 DATA_R0 = 0;
 DATA_G0 = 0;
 DATA_B0 = 0;
 DATA_R1 = 0;
 DATA_G1 = 0;
 DATA_B1 = 0;
 if(ctrl_data_run) begin

  HSYNC   = 1'b1;
  `ifdef BRIGHTNESS_OPERATION
  /*************************************/
  /*  BRIGHTNESS ADDITION OPERATION */
  /*************************************/
  if(SIGN == 1) begin
  // R0
  tempR0 = org_R[WIDTH * row + col   ] + VALUE;
  if (tempR0 > 255)
   DATA_R0 = 255;
  else
   DATA_R0 = org_R[WIDTH * row + col   ] + VALUE;
  // R1
  tempR1 = org_R[WIDTH * row + col+1   ] + VALUE;
  if (tempR1 > 255)
   DATA_R1 = 255;
  else
   DATA_R1 = org_R[WIDTH * row + col+1   ] + VALUE;
  // G0
  tempG0 = org_G[WIDTH * row + col   ] + VALUE;
  if (tempG0 > 255)
   DATA_G0 = 255;
  else
   DATA_G0 = org_G[WIDTH * row + col   ] + VALUE;
  tempG1 = org_G[WIDTH * row + col+1   ] + VALUE;
  if (tempG1 > 255)
   DATA_G1 = 255;
  else
   DATA_G1 = org_G[WIDTH * row + col+1   ] + VALUE;
  // B
  tempB0 = org_B[WIDTH * row + col   ] + VALUE;
  if (tempB0 > 255)
   DATA_B0 = 255;
  else
   DATA_B0 = org_B[WIDTH * row + col   ] + VALUE;
  tempB1 = org_B[WIDTH * row + col+1   ] + VALUE;
  if (tempB1 > 255)
```

```
   DATA_B1 = 255;
  else
   DATA_B1 = org_B[WIDTH * row + col+1    ] + VALUE;
 end
 else begin
 /*************************************/
 /* BRIGHTNESS SUBTRACTION OPERATION */
 /*************************************/
  // R0
  tempR0 = org_R[WIDTH * row + col    ] - VALUE;
  if (tempR0 < 0)
   DATA_R0 = 0;
  else
   DATA_R0 = org_R[WIDTH * row + col    ] - VALUE;
  // R1
  tempR1 = org_R[WIDTH * row + col+1    ] - VALUE;
  if (tempR1 < 0)
   DATA_R1 = 0;
  else
   DATA_R1 = org_R[WIDTH * row + col+1    ] - VALUE;
  // G0
  tempG0 = org_G[WIDTH * row + col    ] - VALUE;
  if (tempG0 < 0)
   DATA_G0 = 0;
  else
   DATA_G0 = org_G[WIDTH * row + col    ] - VALUE;
  tempG1 = org_G[WIDTH * row + col+1    ] - VALUE;
  if (tempG1 < 0)
   DATA_G1 = 0;
  else
   DATA_G1 = org_G[WIDTH * row + col+1    ] - VALUE;
  // B
  tempB0 = org_B[WIDTH * row + col    ] - VALUE;
  if (tempB0 < 0)
   DATA_B0 = 0;
  else
   DATA_B0 = org_B[WIDTH * row + col    ] - VALUE;
  tempB1 = org_B[WIDTH * row + col+1    ] - VALUE;
  if (tempB1 < 0)
   DATA_B1 = 0;
  else
   DATA_B1 = org_B[WIDTH * row + col+1    ] - VALUE;
  end
  `endif
```

```
  /*************************************/
  /*          INVERT_OPERATION          */
  /*************************************/
  `ifdef INVERT_OPERATION
   value2 = (org_B[WIDTH * row + col   ] + org_R[WIDTH * row + col   ]
+org_G[WIDTH * row + col   ])/3;
   DATA_R0=255-value2;
   DATA_G0=255-value2;
   DATA_B0=255-value2;
   value4 = (org_B[WIDTH * row + col+1  ] + org_R[WIDTH * row + col+1  ]
+org_G[WIDTH * row + col+1  ])/3;
   DATA_R1=255-value4;
   DATA_G1=255-value4;
   DATA_B1=255-value4;
  `endif
  /*************************************/
  /******** THRESHOLD OPERATION  ********/
  /*************************************/
  `ifdef THRESHOLD_OPERATION
  value = (org_R[WIDTH * row + col     ]+org_G[WIDTH * row + col
]+org_B[WIDTH * row + col   ])/3;
  if(value > THRESHOLD) begin
   DATA_R0=255;
   DATA_G0=255;
   DATA_B0=255;
  end
  else begin
   DATA_R0=0;
   DATA_G0=0;
   DATA_B0=0;
  end
  value1 = (org_R[WIDTH * row + col+1    ]+org_G[WIDTH * row + col+1
]+org_B[WIDTH * row + col+1   ])/3;
  if(value1 > THRESHOLD) begin
   DATA_R1=255;
   DATA_G1=255;
   DATA_B1=255;
  end
  else begin
   DATA_R1=0;
   DATA_G1=0;
   DATA_B1=0;
  end
  `endif
 end
end

endmodule
```

The "parameter.v" file is also to define paths and names of the input and output file. After processing the image, it is needed to write the processed data to an output image for verifications.

The following Verilog code is to write the processed image data to a bitmap image for verification:

```
/****************** Module for writing .bmp image *************/
/**************************************************************/
module image_write #(parameter
WIDTH = 768, // Image width
HEIGHT = 512, // Image height
INFILE = "output.bmp", // Output image
BMP_HEADER_NUM = 54 // Header for bmp image
)
(
input HCLK, // Clock input
HRESETn, // Reset active low
input hsync, // Hsync pulse
input [7:0] DATA_WRITE_R0, // Red 8-bit data (odd)
input [7:0] DATA_WRITE_G0, // Green 8-bit data (odd)
input [7:0] DATA_WRITE_B0, // Blue 8-bit data (odd)
input [7:0] DATA_WRITE_R1, // Red 8-bit data (even)
input [7:0] DATA_WRITE_G1, // Green 8-bit data (even)
input [7:0] DATA_WRITE_B1, // Blue 8-bit data (even)
output reg Write_Done
);
//-------------------------------------//
//-------Header data for bmp image-----//
//-------------------------------------//
// Windows BMP files begin with a 54-byte header
initial  begin
BMP_header[ 0] = 66;BMP_header[28] =24;
BMP_header[ 1] = 77;BMP_header[29] = 0;
BMP_header[ 2] = 54;BMP_header[30] = 0;
BMP_header[ 3] = 0;BMP_header[31] = 0;
BMP_header[ 4] = 18;BMP_header[32] = 0;
BMP_header[ 5] = 0;BMP_header[33] = 0;
BMP_header[ 6] = 0;BMP_header[34] = 0;
BMP_header[ 7] = 0;BMP_header[35] = 0;
BMP_header[ 8] = 0;BMP_header[36] = 0;
BMP_header[ 9] = 0;BMP_header[37] = 0;
BMP_header[10] = 54;BMP_header[38] = 0;
BMP_header[11] = 0;BMP_header[39] = 0;
BMP_header[12] = 0;BMP_header[40] = 0;
BMP_header[13] = 0;BMP_header[41] = 0;
BMP_header[14] = 40;BMP_header[42] = 0;
BMP_header[15] = 0;BMP_header[43] = 0;
BMP_header[16] = 0;BMP_header[44] = 0;
BMP_header[17] = 0;BMP_header[45] = 0;
BMP_header[18] = 0;BMP_header[46] = 0;
```

```
BMP_header[19] = 3;BMP_header[47] = 0;
BMP_header[20] = 0;BMP_header[48] = 0;
BMP_header[21] = 0;BMP_header[49] = 0;
BMP_header[22] = 0;BMP_header[50] = 0;
BMP_header[23] = 2;BMP_header[51] = 0;
BMP_header[24] = 0;BMP_header[52] = 0;
BMP_header[25] = 0;BMP_header[53] = 0;
BMP_header[26] = 1; BMP_header[27] = 0;
end
//-------------------------------------------------------------//
//------------------Write .bmp file  ----------------------//
//-------------------------------------------------------------//
initial begin
    fd = $fopen(INFILE, "wb+");
end
always@(Write_Done) begin // once the processing was done, bmp image
will be created
    if(Write_Done == 1'b1) begin
        for(i=0; i<BMP_HEADER_NUM; i=i+1) begin
            $fwrite(fd, "%c", BMP_header[i][7:0]); // write the header
        end
        for(i=0; i<WIDTH*HEIGHT*3; i=i+6) begin
  // write R0B0G0 and R1B1G1 (6 bytes) in a loop
            $fwrite(fd, "%c", out_BMP[i  ][7:0]);
            $fwrite(fd, "%c", out_BMP[i+1][7:0]);
            $fwrite(fd, "%c", out_BMP[i+2][7:0]);
            $fwrite(fd, "%c", out_BMP[i+3][7:0]);
            $fwrite(fd, "%c", out_BMP[i+4][7:0]);
            $fwrite(fd, "%c", out_BMP[i+5][7:0]);
        end
    end
end
```

Next, let's write a test bench Verilog code to verify the image processing operations.

```
`timescale 1ns/1ps /*************************************************/
/******* Testbench for simulation ****************/
/*************************************************/
`include "parameter.v" // include definition file module tb_simulation;
//----------------------  Internal Signals -------------------------//
reg HCLK, HRESETn;
wire vsync;
wire hsync;
wire [ 7 : 0] data_R0;
wire [ 7 : 0] data_G0;
wire [ 7 : 0] data_B0;
wire [ 7 : 0] data_R1;
wire [ 7 : 0] data_G1;
wire [ 7 : 0] data_B1;
wire enc_done;
```

```
image_read #(.INFILE(`INPUTFILENAME))
u_image_read
( .HCLK (HCLK ),
.HRESETn (HRESETn ),
 .VSYNC (vsync ),
.HSYNC (hsync ),
.DATA_R0 (data_R0 ),
.DATA_G0 (data_G0 ),
.DATA_B0 (data_B0 ),
.DATA_R1 (data_R1 ),
.DATA_G1 (data_G1 ),
.DATA_B1 (data_B1 ),
.ctrl_done (enc_done)
);
image_write #(.INFILE(`OUTPUTFILENAME))
u_image_write (
.HCLK(HCLK),
.HRESETn(HRESETn),
 .hsync(hsync),
.DATA_WRITE_R0(data_R0),
.DATA_WRITE_G0(data_G0),
.DATA_WRITE_B0(data_B0),
.DATA_WRITE_R1(data_R1),
.DATA_WRITE_G1(data_G1),
.DATA_WRITE_B1(data_B1),
 .Write_Done()
 );
//------------- Test Vectors ---------------//
initial
begin
HCLK = 0;
forever #10 HCLK = ~HCLK;
end
initial
begin
HRESETn = 0;
#25 HRESETn = 1;
end endmodule
```

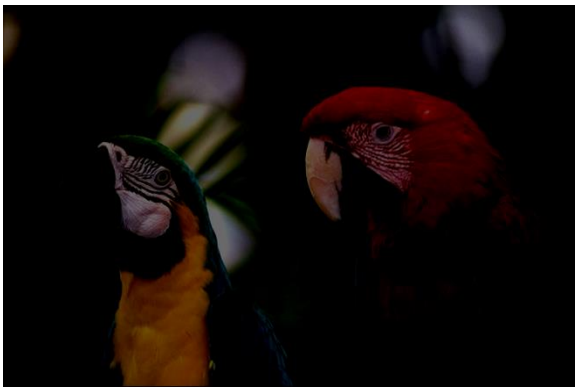
Finally, we have everything to run a simulation to verify the image processing code. Let's use the following image as the input bitmap file:

The image processing operation is selected in the following "parameter.v" file. To change the processing operation, just switch the comment line.

```
/****************************************/
/*********** Definition file ***********/
/************** ***********************************************/
`define INPUTFILENAME "your_image.hex" // Input file name
`define OUTPUTFILENAME "output.bmp" // Output file name
// Choose the operation of code by delete
// in the beginning of the selected line
//`define BRIGHTNESS_OPERATION
 `define INVERT_OPERATION
//`define THRESHOLD_OPERATION
```

CHAPTER – 4 RESULTS OF THE PROJECT

4.1 Brightness Decrease Operation

% VALUE	ORIGINAL IMAGE	OUTPUT IMAGE
50%		
100%		
150%		

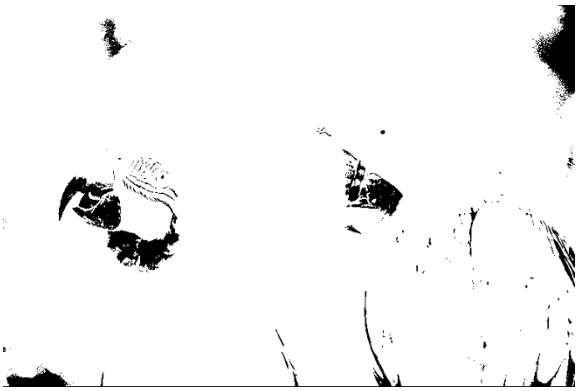
4.2 Brightness Increase Operation

% VALUE	ORIGINAL IMAGE	OUTPUT IMAGE
50%		
100%		
150%		

4.3 Invert Operation

TIMES	ORIGINAL IMAGE	OUTPUT IMAGE
1		
2		

4.4 Threshold Operation

% VALUE	ORIGINAL IMAGE	OUTPUT IMAGE
50%		
90%		
150%		

CONCLUSION

In this project, we undertook the task of implementing image processing on an FPGA using a combination of MATLAB and Verilog. The MATLAB code provided initial processing steps, reading a 24-bit BMP image and converting its RGB data into a hexadecimal file. This file served as the input for the subsequent Verilog implementation. The Verilog code was designed to read this hexadecimal data, process it based on selected operations (brightness addition/subtraction, inversion, thresholding), and then write the processed data to an output BMP file. The Verilog code employed a finite state machine (FSM) to synchronize the image processing steps, including the generation of horizontal and vertical sync pulses.

The integration of MATLAB and Verilog in this project showcased the seamless flow of data from image conversion in MATLAB to FPGA-based processing in Verilog. The modular approach allowed for easy parameterization and flexibility in choosing specific image processing operations. The testbench provided a means to simulate the image processing operations and verify the correctness of the implemented design. Overall, this project demonstrated the successful collaboration between MATLAB and Verilog for image processing on an FPGA, emphasizing the importance of a well-coordinated workflow in embedded systems development.